

Module 1: Introduction & Core Concepts

1.1 What is Git?

Git is a powerful **version control system (VCS)** that constantly tracks every change you make to your files — day or night, 365 days a year.

Git records:

- **What** changed (which lines were edited)
- **When** it changed (timestamp)
- **Who** changed it (author)
- **Where** it happened (which file/directory)

What kind of files can Git track?

Almost **any kind** — not just code:

File Type	Examples
Code	JavaScript, Python, PHP, Java, C++
Text	<code>.txt</code> , <code>.md</code> , <code>.csv</code> , <code>.json</code>
Images	<code>.png</code> , <code>.jpg</code> , <code>.svg</code>
Config	<code>.yaml</code> , <code>.env</code> , <code>.xml</code>
Documents	<code>.html</code> , <code>.css</code>

Key Insight: Git is most commonly used in coding projects, but its power goes far beyond just code.

1.2 What is Version Control?

Version control is the practice of **tracking and managing changes** to files over time.

Why do we need it?

Imagine this real-world scenario:

Day	What Happens
Day 1	You write code → Client loves it ✅
Day 30	Client requests changes → You update the code ✅
Day 35	Client says "Actually, the OLD version was better" 🤔

Without Git: You've already overwritten the original. The old version is **gone forever**.

With Git: Every version is safely stored. You can **roll back** to any previous state in seconds.

This is why Git is called a Version Control System — it keeps every version of your files safely stored so you can go back to any previous state whenever you need to.

Key capabilities:

- ✅ Save **multiple versions** of the same file
- ✅ **Roll back** to any previous version instantly
- ✅ Never lose a file or accidentally overwrite something important
- ✅ Track the complete **history** of every change

1.3 A Brief History

Git was created by **Linus Torvalds** — the same brilliant mind behind **Linux**.

"Most tools we programmers use have a short lifespan, but Git is one of those rare ones that once you learn it, stays useful for the rest of your career."

Git is:

- **Simple** — built around a few core commands and concepts
- **Powerful** — handles projects of any size
- **Universal** — used by virtually every software team in the world

1.4 Git vs. GitHub

This is one of the most important distinctions to understand:



Git is the coffee.



GitHub is the coffee shop where that coffee is served.

They are **connected** but **completely different**.

```
graph TD
    subgraph Git ["🖥️ Git (Local Version Control)"]
        LocalFiles["Local Files & Folders"]
        LocalHistory["Track Change History (Hidden .git)"]
        CLI["Runs in Terminal (Local PC)"]
    end

    subgraph GitHub ["☁️ GitHub (Cloud Hosting Platform)"]
        RemoteRepos["Remote Repositories"]
        Collaboration["Team Collaboration (PRs, Issues)"]
        WebUI["Web Interface (Hosting & Sharing)"]
    end

    LocalHistory -->|"git push (Upload)"| RemoteRepos
    RemoteRepos -->|"git clone / pull (Download)"| LocalFiles

    style Git fill:#f5f5f5,stroke:#333,stroke-width:2px,color:#000
    style GitHub fill:#e8f4f8,stroke:#0288d1,stroke-width:2px,color:#000
```

Feature	Git	GitHub
What is it?	A version control tool	A cloud hosting platform
Where does it run?	Locally on your computer	On the internet (cloud)
Purpose	Track changes to files	Share, collaborate, and back up code
Created by	Linus Torvalds	Chris Wanstrath, Tom Preston-Werner
Owned by	Open-source community	Microsoft
Alternatives	Mercurial, SVN	GitLab, Bitbucket
Visual State	Offline tracking	Online storage and UI

Other platforms similar to GitHub:

- **GitLab** — widely used, offers CI/CD pipelines
- **Bitbucket** — popular with Atlassian (Jira) users

GitHub remains the most popular and widely adopted platform, now owned by Microsoft with a strong focus on the open-source community.

1.5 Git Architecture: Local vs. Remote

Git is divided into **two major parts**:

-  **Local (Your Computer)** — where you do all your work
-  **Remote (The Cloud)** — where you push/upload your work (usually GitHub)

The Four Stages of Git

```
graph LR
  WD["🖥️ Working Directory<br/>(Local files, untracked/modified)"]
  SA["📋 Staging Area<br/>(Prepared index checklist)"]
  LR["💾 Local Repository<br/>(Committed history in .git)"]
  RR["☁️ Remote Repository<br/>(Hosted on GitHub/GitLab)"]

  WD -->|"git add"| SA
  SA -->|"git commit"| LR
  LR -->|"git push"| RR
  RR -->|"git clone / fetch"| WD
  LR -->|"git checkout / switch"| WD

  style WD fill:#fff9c4,stroke:#fbc02d,stroke-width:2px,color:#000
  style SA fill:#e1f5fe,stroke:#0288d1,stroke-width:2px,color:#000
  style LR fill:#e8f5e9,stroke:#388e3c,stroke-width:2px,color:#000
  style RR fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px,color:#000
```

Explanation of each stage:

Stage	What happens	Analogy
Working Directory	You write code, create/modify files	Your desk where you work
Staging Area	You mark changes as "ready"	A checklist before packing a suitcase
Local Repository	You permanently save (commit) changes	Locking the suitcase
Remote Repository	You upload to the cloud (push)	Shipping the suitcase to storage

Detailed Step-by-Step Architecture Flow:

Step	Command	Stage Shift	Local Status	Git Internal Functionality
1. Create / Edit Files	(Manual edit)	Working Directory	Files are Untracked or Modified locally	Changes exist only in memory or transient files on disk. Git does not record history yet.
2. Stage Changes	<code>git add <file></code>	Working Directory → Staging Area	Files are Staged (ready to commit)	Git creates a Blob object containing the file content, compresses it, and updates the <code>index</code> binary file in <code>.git/</code> .
3. Commit Staged Files	<code>git commit -m "msg"</code>	Staging Area → Local Repository	Files are Committed to local history	Git creates a Tree object (representing directory state) and a Commit object (meta-data: author, parent link, tree hash) inside <code>.git/objects/</code> . Moves the local branch reference (e.g. <code>main</code>) to this new commit.
4. Push to Cloud	<code>git push origin <branch></code>	Local Repository → Remote Repository	Local commits match Remote state	Git negotiates with the remote server, packs the new objects, uploads them via SSH/HTTPS, and updates the remote-tracking ref (e.g., <code>origin/main</code>).

Think of GitHub like Google Drive for code — you *could* keep everything only on your device, but you store it in the cloud so you can access it from anywhere. Even if something gets deleted locally, it stays safe online.

What is a Repository?

A **repository** (or "repo") is a place where all the versions of your files and their complete change history are stored.

- **Local repository** — a specific folder on your own computer (contains a hidden `.git` folder)
- **Remote repository** — lives on a cloud server like GitHub

Think of a repository as a **digital cabinet for your code** — a secure place where Git neatly stores every record of your work.

Inside the repository, Git automatically creates system files that track:

- Your changes
- History
- Commits
- Branch information
- And more

These files are managed **entirely by Git itself**.



Key Takeaways

1. **Git** = local version control tool; **GitHub** = cloud hosting platform
 2. Git tracks **what, when, who, and where** for every change
 3. Version control means **never losing** a previous version of your work
 4. Git architecture flows: **Working Directory** → **Staging Area** → **Local Repo** → **Remote Repo**
 5. A repository is where Git stores all versions and history of your project
-